Universidad Nacional Mayor de San Marcos · Facultad de Ingeniería Electrónica y Eléctrica

Guion de exposición

Número real en memoria y resistencia por colores (Java): estructura, tiempos, qué decir, demostraciones y preguntas frecuentes

> **Duración sugerida:** 15 a 20 minutos
>
> **Integrantes:** Pensado para 3 (con variantes para 2 o 4)
>
> **Incluye:** Diapositivas sugeridas, guion, datos de prueba, preguntas frecuentes y plan B

- Estructura y tiempos
- Checklist previo
- Diapositivas sugeridas
- Qué decir en cada bloque
- Demostraciones
- Cómo explicar Java
- Datos de prueba
- Preguntas frecuentes
- Plan B
- Tarjeta de bolsillo

Material de estudio no oficial. Los programas fueron compilados y ejecutados; las salidas mostradas son reales. Los programas de la resistencia se verificaron con todas sus combinaciones de colores (1 000 y 1 200).

Índice

Guion de exposición 1
1. Objetivo y público 1
2. Estructura y tiempos 1
3. Antes del día de la exposición 2
4. Diapositivas sugeridas 2
5. Guion: qué decir en cada bloque 4
6. Cómo explicar Java a quien no sabe Java 7
7. Datos de prueba (para tener impresos) 8
8. Errores que conviene evitar al exponer 8
9. Preguntas frecuentes (y cómo responderlas) 9
10. Plan B si la demostración falla 10
11. Tarjeta de bolsillo 10

Guion de exposición

1. Objetivo y público

Objetivo: que al terminar, los compañeros puedan responder dos preguntas:

1. ¿Cómo guarda la computadora un número con decimales (y por qué a veces «se equivoca»)?
2. ¿Cómo funciona un programa que, a partir de los **colores** de una resistencia, da **el número de cada banda** (y, como ampliación, su **valor**)?

Público: compañeros que vieron algo de C++ pero **no saben Java**. Por eso, la estrategia de toda la exposición es:

> **Primero la idea (con un dibujo), después el código, y siempre comparando con C++.**

2. Estructura y tiempos

Pensado para **15 a 20 minutos** y un grupo de **3 personas** (A, B y C). Si son 2, A hace los bloques 1–3 y B los bloques 4–8; si son 4, el cuarto integrante hace las dos demostraciones.

N.°	Bloque	Tiempo	Quién	Material
1	Apertura y pregunta gancho	1 min	A	Diapositiva «¿0.1 + 0.2 = 0.3?»
2	Parte 1: cómo se guarda un número real	5 min	A	Figuras de bits, binario, IEEE 754 y ejemplos
3	Demostración 1 (programa de números reales)	2 min	A o C	NetBeans: `-6.75`, `0.1` y curiosidades
4	Parte 2: resistencias y código de colores	2 min	B	Tabla de colores y dibujo de la resistencia
5	Parte 2: el programa de clase y el nuestro	3–4 min	B	Comparación, tabla C++ ↔ Java, código clave, diagrama de flujo
6	Demostración 2 (números de las bandas y ampliación)	2–3 min	C	NetBeans: lista de casos de prueba
7	Conexión entre las dos partes	1 min	C	El ejemplo 12 × 0.1
8	Conclusiones y preguntas	2–3 min	Todos	Resumen en 3 ideas

> **Consejo**
>
> Si el profesor da menos tiempo, recorta los bloques 2 y 5 (no las demostraciones): una demo que funciona convence más que cualquier diapositiva.

3. Antes del día de la exposición

- ☐ **Probar los programas en la computadora donde se va a exponer.** Si es otra PC, abrir los proyectos un día antes: la primera vez Maven necesita internet para descargar sus componentes.
- ☐ **Agrandar la letra** del editor y de la ventana *Output* de NetBeans (*Tools → Options → Fonts & Colors* y *Tools → Options → Miscellaneous → Output*) para que se lea desde el fondo del salón.
- ☐ Tener **impresa la lista de datos de prueba** (sección 7) para no improvisar.
- ☐ Tener un **plan B**: este PDF abierto en las ejecuciones reales (o capturas de pantalla) por si NetBeans falla.
- ☐ Cerrar programas y notificaciones; poner la PC en modo «no molestar».
- ☐ **Ensayar dos veces con cronómetro**, cada integrante con su parte.
- ☐ Repartir las preguntas frecuentes (sección 9): cada uno debe dominar las de su parte.

4. Diapositivas sugeridas

Todas las figuras están en la carpeta `diagramas/` (archivos `.svg`, se pueden insertar en PowerPoint o Google Slides).

N.°	Título	Contenido	Figura
1	Portada	Título, curso, integrantes	—
2	¿0.1 + 0.2 = 0.3?	La pregunta gancho y la respuesta de la computadora	—
3	Bits, bytes y memoria	Bit, byte, direcciones; `float` = 4 bytes, `double` = 8	`memoria_bytes.svg`
4	De decimal a binario	Divisiones (parte entera) y multiplicaciones (parte fraccionaria)	`conversion_binario.svg`
5	Punto flotante	Notación científica en base 2	`notacion_cientifica.svg`
6	Estándar IEEE 754	Signo, exponente (con sesgo) y mantisa en `float` y `double`	`ieee754_formatos.svg`
7	Ejemplo: −6.75	Los 32 bits y su lectura: C0D80000	`ieee754_ejemplo_menos_6_75.svg`
8	Ejemplo: 0.1	Binario infinito, redondeo y valor guardado	`ieee754_ejemplo_0_1.svg`
9	Demostración 1	Captura del programa (por si se necesita)	—
10	La resistencia y su código de colores	Qué es, por qué colores, tabla y regla para recordar	`tabla_colores.svg`, `resistencia_bandas.svg`
11	Programa de clase vs. el nuestro	Número → color vs. color → número	`comparacion_programas.svg`
12	Java en 1 minuto (para quien sabe C++)	Tabla de equivalencias reducida	—
13	El código clave	`toLowerCase`, `switch` con texto y la salida `4 - 7 - 2`; en la ampliación, `Math.pow`	—
14	Diagrama de flujo	El programa completo de un vistazo	`flujo_numeros_bandas.svg` (ampliación: `flujo_resistencia.svg`)
15	Demostración 2	Casos de prueba	—
16	Conclusiones	Las 3 ideas	—
17	¿Preguntas?	—	—

Tabla reducida para la diapositiva 12:

C++	Java
`cout << "Hola";`	`System.out.print("Hola");`
`cin >> color;`	`color = datos.next();`
`string`	`String`
`pow(10, 2)`	`Math.pow(10, 2)`
`if`, `switch`, `while`	¡Iguales!

5. Guion: qué decir en cada bloque

Las frases entre comillas son una guía; no hay que memorizarlas, pero sí la **idea** de cada una.

Bloque 1 — Apertura (A, 1 min)

> «Buenos días. Empezamos con una pregunta: si le pedimos a la computadora que sume 0.1 más 0.2, ¿qué creen que responde? … Responde **0.30000000000000004**. No es un error de Java: pasa en casi todos los lenguajes. Hoy vamos a explicar **por qué**, es decir, **cómo se guarda un número real en la memoria**. Y después les mostraremos un programa en Java que, a partir de los **colores de una resistencia**, da **el número de cada banda**, y una ampliación que calcula su valor, donde verán que las dos partes están conectadas.»

Bloque 2 — Parte 1: el número real en memoria (A, 5 min)

Diapositiva 3 (bits y bytes):

> «La computadora solo guarda ceros y unos: los **bits**. Ocho bits forman un **byte**. La memoria RAM es como una fila enorme de casilleros de un byte, y cada casillero tiene un número, su dirección. Un `float` ocupa 4 casilleros y un `double`, 8.»

Diapositiva 4 (binario):

> «Para guardar 6.75, primero lo pasamos a binario. La parte entera, 6, se divide entre 2 y se leen los restos de abajo hacia arriba: 110. La parte decimal, 0.75, se multiplica por 2 y se toman las partes enteras: 0.11. Entonces 6.75 es **110.11** en binario.»

> «Pero con 0.1 pasa algo: al multiplicar por 2 nunca llegamos a cero; el patrón 0011 se repite **para siempre**. Es como 1/3 en decimal, 0.333… infinito. La memoria es finita, así que **hay que cortar y redondear**. Ahí nace el error.»

Diapositiva 5 (punto flotante):

> «Para guardar números muy grandes y muy pequeños se usa la **notación científica**, como en física: 6 750 000 es 6.75 por 10 a la 6. La computadora hace lo mismo pero en base 2: 110.11 es **1.1011 por 2 al cuadrado**. Como el punto se mueve, se llama **punto flotante**: de ahí viene el nombre `float`.»

Diapositiva 6 (IEEE 754):

> «Todas las computadoras siguen el estándar **IEEE 754**. Un `float` tiene 32 bits: **1 bit de signo**, **8 de exponente** y **23 de mantisa**. Dos trucos: el exponente se guarda **sumándole 127** (el *sesgo*), para poder tener exponentes negativos; y el «1.» del inicio **no se guarda**, porque en binario siempre es 1. El `double` es igual pero con 64 bits: 1, 11 y 52, y sesgo 1023.»

Diapositiva 7 (−6.75):

> «Armemos −6.75: es negativo, así que el signo es 1. Exponente: 2 más 127 es 129, en binario 10000001. Mantisa: lo que va después del «1.», o sea 1011 y el resto ceros. Juntando todo, en hexadecimal: **C0D80000**. Y si lo leemos al revés con la fórmula, da exactamente −6.75.»

Diapositiva 8 (0.1):

> «Con 0.1, la mantisa tiene que cortarse en 23 bits y el último se redondea. Por eso, lo que realmente queda guardado es 0.100000001490116… **No es exactamente 0.1.** Cuando sumamos 0.1 y 0.2, esos pequeños errores se juntan y aparece el 0.30000000000000004. Conclusión práctica: **los reales no se comparan con `==`**, se compara con una tolerancia.»

Bloque 3 — Demostración 1 (2 min)

1. Abrir `Numero_Real_en_Memoria` y presionar `F6`.
2. Escribir `1` y luego `-6.75`.

> «Miren: el programa hace la misma conversión que hicimos en la pizarra, paso a paso: las divisiones, las multiplicaciones, el exponente con sesgo y la mantisa. Y abajo compara con los bits que **Java realmente guarda**: **coinciden**. También muestra el hexadecimal C0D80000 y cómo quedan los 4 bytes en la memoria.»

3. Escribir `1` y luego `0.1`.

> «Ahora 0.1: la fracción nunca llega a cero, el último bit se redondea, y el valor exacto guardado no es 0.1.»

4. Escribir `2` (curiosidades).

> «Y aquí el 0.1 + 0.2, y el detalle que conecta con la segunda parte: 12 por 0.1 da 1.2000000000000002.»

Bloque 4 — Resistencias y código de colores (B, 2 min)

> «Una **resistencia** es un componente que se opone al paso de la corriente; se mide en **ohmios**. Como son tan pequeñas, en vez de números se pintan **bandas de colores**. Cada color es un número del 0 al 9. Para recordar el orden: ***Ni Me Robes Nada, Amigo; Ve Al Viejo Gran Banco***: Negro, Marrón, Rojo, Naranja, Amarillo, Verde, Azul, Violeta, Gris, Blanco.»

> «Se lee así: la 1.ª y 2.ª banda son las cifras; la 3.ª dice por cuánto multiplicar; la 4.ª es la tolerancia. Amarillo-violeta-rojo es 4, 7 y por 100: **4 700 ohmios, o 4.7 kilo-ohmios**.»

Bloque 5 — El programa de clase y el nuestro (B, 3–4 min)

Diapositiva 11:

> «En clase vimos un programa que recibe **números** y muestra los **colores**. La tarea era hacer lo contrario: que el usuario escriba **los nombres de los colores** y el programa dé **el número de cada banda**: amarillo, violeta, rojo → 4, 7, 2. Mantuvimos la misma estructura del profesor para que se pueda comparar: es el mismo `switch`, pero al revés. Además hicimos una ampliación que, con esos números, calcula el valor de la resistencia.»

Diapositiva 12 (Java para quien sabe C++):

> «Si no conocen Java, no se preocupen: es muy parecido a C++. `System.out.print` es como `cout`; `Scanner` es como `cin`; `String` es `string`. Y `if`, `switch` y `while` funcionan **igual**. Una advertencia: en las capturas de NetBeans aparecen cosas como «s:» o «a:» en gris; eso **no es código**, es una ayuda que dibuja NetBeans con el nombre de los parámetros.»

Diapositiva 13 (el código clave) — mostrar solo estos fragmentos, con letra grande:

```
colorb1 = datos.next().toLowerCase();   // lee "Amarillo" y lo deja "amarillo"

switch(colorb1)
{
    case "amarillo": ncb1 = 4;
                     break;
    // ... un case por cada color
    default: sw = 1;                    // color que no existe
}

System.out.print("\nNumeros de las bandas: " + ncb1 + " - " + ncb2 + " - " + ncb3);
```

> «Primero leemos cada color y lo pasamos a minúsculas, así da igual si escriben ROJO o Rojo. Después, con un `switch` convertimos cada nombre en su número; en Java el `switch` puede comparar textos. Si alguien escribe un color que no existe, entra al `default` y marcamos un error con la variable `sw`. Al final, si no hubo errores, mostramos los tres números: 4 - 7 - 2.»

Si hay tiempo o preguntan por el valor (ampliación):

```
double vr = ncb1 * 10 + ncb2;           // 4 y 7 -> 47
vr = vr * Math.pow(10, ncb3);           // 47 x 10^2 = 4700
```

> «Con esos mismos números, la ampliación aplica la fórmula: la primera cifra por 10 más la segunda, por 10 elevado al multiplicador, con `Math.pow`.»

Diapositiva 14 (diagrama de flujo):

> «Este es el programa completo: un ciclo que pide los 3 colores, los convierte, verifica que sean válidos, muestra los números y pregunta si desea terminar.»

Bloque 6 — Demostración 2 (C, 2 min)

Abrir `Numeros_bandas_segun_Colores_Resistencia.java`, clic derecho → *Run File* (`Shift` + `F6`), e ingresar en este orden:

Ingresar	Debe mostrar	Qué comentar
`amarillo`, `violeta`, `rojo`	`Numeros de las bandas: 4 - 7 - 2`	El ejemplo de la diapositiva
`marron`, `NEGRO`, `Naranja`	`Numeros de las bandas: 1 - 0 - 3`	Acepta mayúsculas
`celeste`, `negro`, `dorado`	Dos mensajes de color no válido	Validación con `default` (el dorado no es una cifra)
`0`	`****** FIN *******`	Termina el ciclo

Ampliación (si hay tiempo o preguntan por el valor): `F6` ejecuta `Valor_Resistencia_segun_Colores_Bandas`:

Ingresar	Debe mostrar	Qué comentar
`amarillo`, `violeta`, `rojo`	`4700.0 Ohm` · `4.7 kOhm`	47×10^2
`marron`, `rojo`, `dorado`	`1.2 Ohm`	El dorado multiplica por 0.1
`amarillo`, `violeta`, `violeta`	`4.7E8 Ohm` · `470.0 MOhm`	`E8` es notación científica: $\times 10^8$
`0`	`****** FIN *******`	Termina el ciclo

Bloque 7 — Conexión entre las partes (C, 1 min)

> «¿Recuerdan el 0.1 que no se puede guardar exacto? En la ampliación que calcula el valor, la banda dorada multiplica por 0.1, y 12 por 0.1 da **1.2000000000000002**. Por eso, en ese programa, en lugar de multiplicar por 0.1 **dividimos entre 10**, y sale 1.2 limpio. Lo comprobamos con las 1 200 combinaciones posibles de colores: todas dan el valor correcto.»

Bloque 8 — Conclusiones (todos, 1 min + preguntas)

> «Tres ideas para llevarse:
>
> 1. Los números reales se guardan como **signo, exponente y mantisa** (IEEE 754), y muchos decimales como 0.1 **no son exactos** en binario.
> 2. Por eso los reales no se comparan con `==` y a veces conviene dividir en lugar de multiplicar por un decimal.
> 3. Con un `switch` convertimos los **nombres de los colores** en **el número de cada banda**; con esos números, la ampliación calcula el valor de la resistencia. Muchas gracias. ¿Preguntas?»

6. Cómo explicar Java a quien no sabe Java

1. **Compara siempre con C++**: «esto es como `cout`», «esto es como `cin`».
2. **Diagrama antes que código**: primero el diagrama de flujo o la figura; después el fragmento de código.
3. **No leas el programa completo**: muestra solo las líneas importantes (lectura, `switch` y lo que se muestra), con la letra bien grande.
4. **Señala con el cursor** la línea de la que hablas y di su número.

5. **Evita palabras técnicas** innecesarias (JVM, bytecode, Maven). Si alguien pregunta, una frase basta: «Java se traduce a un código intermedio que funciona en cualquier sistema».
6. **Explica lo gris de NetBeans** (`s:` , `a:`) al inicio, para que nadie se confunda.
7. **Usa ejemplos concretos**: «amarillo es 4, violeta es 7», no «la variable `ncb1` toma el valor del dígito».

7. Datos de prueba (para tener impresos)

Programa 1 (números reales): `1` → `-6.75` · `1` → `0.1` · `1` → `4700` · `2` (curiosidades) · `0` (salir).

Programa 2 (números de las bandas): amarillo violeta rojo · marron NEGRO Naranja · celeste negro dorado · `0`.

Ampliación (valor de la resistencia): amarillo violeta rojo · marron rojo dorado · amarillo violeta violeta · `0`.

8. Errores que conviene evitar al exponer

- Leer las diapositivas o el código palabra por palabra.
- Poner mucho texto en una diapositiva (máximo 5 líneas o una figura).
- Hacer la demostración sin haberla probado en esa computadora.
- Usar números o colores improvisados en la demo (usa la lista de la sección 7).
- Hablar mirando la pantalla en vez de al público.
- Pasarse del tiempo: ensaya con cronómetro.

9. Preguntas frecuentes (y cómo responderlas)

Pregunta	Respuesta corta
¿Por qué usan `double` y no `float`?	`double` tiene el doble de bits y unas 16 cifras de precisión (el `float`, unas 7). Es el tipo por defecto para decimales en Java y C++.
¿Qué pasa si escribo el color con mayúsculas?	Funciona: `toLowerCase()` lo convierte a minúsculas antes del `switch`.
¿Y si escribo «marrón» con tilde?	Hay que escribirlo sin tilde (`marron`), como en el programa del profesor; con tilde aparece como color no válido.
¿Por qué `switch` y no `if`?	Ambos sirven; el `switch` es más claro para comparar una variable con muchos valores fijos, y es lo que usó el profesor. Java permite `switch` con textos desde la versión 7.
¿Por qué no muestran el valor de la resistencia?	La tarea pide el número de cada banda. El valor lo calcula la ampliación (`Valor_Resistencia_segun_Colores_Bandas`) con la fórmula $(\text{cifra}_1 \times 10 + \text{cifra}_2) \times 10^{\text{multiplicador}}$.
¿Por qué la 3.ª banda no acepta dorado?	Porque el dorado y el plateado no son cifras del 0 al 9: solo sirven como multiplicador (×0.1, ×0.01) o tolerancia. La ampliación sí los acepta en la 3.ª banda.
¿Qué hace `Math.pow(10, ncb3)` (en la ampliación)?	Calcula 10 elevado a `ncb3`: 10^2, 10^3… Es el multiplicador.
¿Por qué sale `4.7E8`?	Es notación científica de Java: 4.7×10^8. Por eso también mostramos `470.0 MOhm`.
¿Por qué dividen entre 10 en vez de multiplicar por 0.1?	Porque 0.1 no es exacto en binario: 12 × 0.1 = 1.2000000000000002, mientras que 12 / 10 = 1.2.
¿Qué es la variable `sw`?	Una «bandera»: empieza en 0 y se pone en 1 si algún color no es válido; solo se muestra el resultado si sigue en 0.
¿Qué es el sesgo?	Un número que se suma al exponente (127 en `float`) para guardar exponentes negativos sin usar signo.
¿Qué es el bit implícito?	El «1.» inicial de la mantisa: siempre es 1, por eso no se guarda y se gana un bit de precisión.
¿Por qué 0.1 + 0.2 no da 0.3?	Ni 0.1 ni 0.2 se guardan exactos (son infinitos en binario); sus pequeños errores se suman.
¿Qué es little-endian?	Que la PC guarda primero el byte menos significativo: −6.75f (C0 D8 00 00) queda en memoria como 00 00 D8 C0.
¿Por qué el programa de clase no muestra nada con 5, 6 o 7?	Sus `switch` no tienen los casos verde, azul y violeta; caen en `default` y no se muestra ningún mensaje. Nuestro programa incluye los 10 colores y avisa los errores.
¿Se puede considerar la tolerancia?	Sí: la versión mejorada lee la 4.ª banda y muestra el valor mínimo y máximo.
¿Qué pasa si escribo una letra en «Desea terminar»?	Nada malo: se lee como texto; cualquier respuesta distinta de `0` continúa.
¿Qué significa `public static void main(String[] args)`?	Es el punto de inicio del programa, como `int main()` en C++.
¿Qué son «s:» y «a:» en NetBeans?	Ayudas visuales con el nombre de los parámetros; no forman parte del código.

10. Plan B si la demostración falla

1. Mantener la calma: «Mientras se soluciona, les muestro la ejecución que hicimos antes».
2. Abrir la guía completa en la sección de **ejecución real** de cada programa (Parte 1: sección 13.2; Parte 2: sección 5.7, y 6.7 para la ampliación) o mostrar capturas tomadas el día anterior.
3. Explicar con la **prueba de escritorio** (Parte 2, sección 5.9): se puede seguir a mano en la pizarra.

11. Tarjeta de bolsillo

Dato	Valor
`float`	32 bits: signo 1 · exponente 8 (sesgo 127) · mantisa 23 · ~7 cifras
`double`	64 bits: signo 1 · exponente 11 (sesgo 1023) · mantisa 52 · ~16 cifras
Fórmula	valor = (−1)^s × 1.mantisa × 2^(exponente − sesgo)
−6.75	$110.11_2 = 1.1011 \times 2^2$ → `C0D80000` (exacto)
0.1	$0.000110011\ldots_2$ (infinito) → `3DCCCCCD` → 0.100000001490116…
Colores 0–9	Negro, Marrón, Rojo, Naranja, Amarillo, Verde, Azul, Violeta, Gris, Blanco
Números de las bandas	amarillo-violeta-rojo → 4 - 7 - 2
Fórmula resistencia	$(\text{cifra}_1 \times 10 + \text{cifra}_2) \times 10^{\text{multiplicador}}$
Ejemplo	amarillo-violeta-rojo = 47 × 100 = 4 700 Ω = 4.7 kΩ
NetBeans	`F6` ejecutar proyecto · `Shift` + `F6` ejecutar archivo